

Plan Módulo Integración SAP

MagEAM ↔ S/4HANA · Análisis técnico y plan de construcción · David Cabezas (VSC) · 2026-05-14

Contexto

En la reunión del 2026-05-13 con el cliente (Bruno Soto, Reynaldo Martínez), la principal barrera técnica identificada fue la **integración bidireccional con SAP**, especialmente la **escritura de datos**. El cliente migrará a S/4HANA fin de año o el próximo. La gestión SAP es centralizada (hoy en Perú, pronto global) con políticas estrictas de ciberseguridad.

Este documento analiza el código SAP ya presente en MagEAM (~1.700 líneas escritas), identifica el TODO crítico que falta, y propone un plan incremental en 7 etapas para tener un módulo funcional listo para el piloto en Azure del cliente.

1 · Lo que YA está construido (no se toca, se completa)

Inventario del código SAP existente en el repo AMS-Production. La base es sólida; falta principalmente el **transporte real** de los payloads outbound.

Componente	LOC	Estado	Función
sap_rfc_connector.py	1.041	■ Esqueleto + dry-run	Connector pyrfc BAPI/RFC clásicos
sap_sync_service.py	136	■ build_payload OK · transporte TODO	Compuempote PENDING/SENT/ACKED
routers/sap_pm.py	117	■ Funcional	Endpoints LECTURA: plans, BOM, measuring, permits, cost centers
routers/sap.py	78	■ Funcional	Upload packages + state machine
agentic_sap_sync_service.py	235	■ Esqueleto	Sync agentic
sap_service.py	82	■ Mock reader	Lee mocks IE03/IW38/IP10/MM60/IL03
sap_mock/data/*.json	—	■ JSON realista	equipment_master, FL, BOM, work_orders, plans
frontend: SapPmPage / SAPReview		■ UI funcional	Páginas de revisión SAP

Modelos DB ya creados: SapSyncLogModel (cola outbound), SAPUploadPackageModel (paquetes), MaintenancePlanModel, BOMItemModel, MeasuringPointModel, WorkPermitModel, CostCenterModel. La data y la estructura está. Lo que falta es el **transporte que efectivamente llame a SAP**.

2 · El TODO crítico — corazón del problema

En `api/services/sap_sync_service.py` línea 132, el código actual tiene este comentario:

```
# def _transport_send(payload: dict) -> tuple[bool, str|None, str|None]:
#     """Actually call SAP. Return (ok, sap_ref, error_message)."""
#     raise NotImplementedError(
#         "Plug the real SAP transport here (RFC / IDoc / REST)."
```

Hoy, los payloads se generan y se ponen en la cola `sap_sync_log` con status PENDING. **Nadie los procesa**. Resolver ese transporte = módulo SAP funcional.

3 · Plan de construcción en 7 etapas

Etapa 1 · Strategy Pattern para transporte intercambiable

3-4 días

Crear interfaz abstracta `SAPTransport` con 4 implementaciones bajo `api/services/sap_transports/`: `dry_run` (default), `rfc` (BAPI vía `pyrfc`, reusa código existente), `odata` (REST S/4HANA NUEVO), `agent_ui` (Playwright sobre SAP GUI NUEVO, Plan B mencionado por José). El selector lee `SAP_TRANSPORT` de env. En QA/dev no se toca SAP real; en piloto se cambia una env var.

Etapa 2 · Mock-SAP en Docker

2 días

Convertir `sap_mock/` en un servicio Docker `ocp-sap-mock` con FastAPI que expone endpoints OData realistas: `API_MAINTNOTIFICATION`, `API_MAINTORDER`, `API_EQUIPMENT`. Permite desarrollar el conector sin SAP real. Cuando lleguen credenciales del cliente, se cambia una env var y listo.

Etapa 3 · Worker que procesa la cola

2 días

Worker async en `api/workers/sap_worker.py` que cada N segundos lee PENDING, llama `_transport_send()`, marca SENT/ERROR. Retry exponencial con back-off, dead letter después de 3 fallos. Endpoint admin `/api/v1/sap/queue` para ver estado en vivo.

Etapa 4 · Conector OData S/4HANA

5-7 días

Implementación nueva en `sap_transports/odata.py`. Auth OAuth 2.0 o Basic + CSRF token. CRUD para Notification, Order, Operation, Component, Confirmation. Soporte de \$batch para enviar OT + operaciones + materiales en una sola llamada. Validación contra mock antes de pegarle al SAP cliente.

Etapa 5 · Agente UI (Plan B José)

7-10 días

Solo si Plan A (OData) es bloqueado por IT/Seguridad del cliente. Container con Playwright + SAP GUI for HTML/Fiori. Cuenta de servicio con privilegios mínimos. Scripts para IE01 (crear equipo), IW21 (crear aviso), IW31 (crear OT). Más lento pero funciona aunque la API esté bloqueada.

Etapa 6 · Tests de contrato

3 días

Schemathesis sobre mock OData → garantiza que el conector cumple el contrato OpenAPI declarado. Pact contracts (ticket SF-707) → consumidor (MagEAM) vs proveedor (SAP). Snapshot tests del payload BAPI generado.

Etapa 7 · UI de monitoreo SAP Sync

3 días

Nueva página `/sap-sync`: cola en tiempo real (PENDING/SENT/ERROR), logs por intento, reintento manual, switch de transporte (admin only), métricas (tasa éxito, latencia media, errores por tipo).

4 · Timeline y costo total

Bloque	Etapas	Duración	Resultado
MVP demo-able	1 + 2 + 3	8-10 días	Módulo funcional con Mock-SAP. Demo de integración para cliente sin r
Conector real	4	5-7 días	Cliente acepta API OData → integración productiva contra S/4HANA cli
Fallback IT-bloqueado	5	7-10 días (sólo si aplica)	IT del cliente bloquea API, agente UI Playwright sobre SAP GUI.
Calidad + Operación	6 + 7	6 días	Tests de contrato (cierra ticket SF-707) + UI de monitoreo en vivo.
TOTAL	1-7	25-30 días	Módulo SAP completo, productivo, monitoreable.

El MVP (etapas 1-3, 8-10 días) ya es vendible: tenés Mock-SAP funcionando para mostrar al cliente que la integración es real, ANTES de tener acceso a su sistema SAP. Las etapas 4-7 son para el piloto productivo en Azure del cliente.

5 · Ideas para diferenciar y acelerar

- Replicar contratos SAP en TypeScript

El frontend conoce la forma exacta del payload y valida antes de enviar — menos round-trips al backend.

- Modo "shadow" de auditoría

Cada operación se envía a SAP y se loguea en MagEAM con el diff → trazabilidad total de qué entró efectivamente al SAP del cliente.

- Dashboard de cobertura SAP por cliente

Pantalla que muestra qué módulos SAP están conectados, qué BAPIs usados, % cobertura. Útil para vender el avance al cliente.

- Plantillas por cliente

Cada cliente tiene su *sap_config.yaml* con mapping de campos (porque cada SAP es ligeramente distinto). Onboarding rápido a clientes nuevos.

- Visor de IDocs inbound

SAP puede empujar IDocs a MagEAM (status, confirmaciones de OT cerrada). Armar webhook inbound = visión completa bidireccional.

- Métricas comerciales integradas

Cuántas OTs sincronizadas, cuánto tiempo ahorrado vs entrada manual a SAP. Datos crudos para argumentar ROI al cliente.

6 · Recomendación de arranque

Esta semana: Etapas 1 + 2 + 3 (MVP de 8-10 días).

Al finalizar esa semana y media:

- Tenemos un módulo SAP **funcionando end-to-end** contra Mock-SAP.
- Podemos mostrar al cliente que **la integración no es vaporware**.
- El equipo de IT del cliente puede inspeccionar el código y los contratos OData ANTES de discutir accesos.
- Reduce el riesgo del workshop con Andrea/Jorge Alquinta: vamos con la demo lista, no con promesas.
- Las etapas 4-7 quedan para arrancar después del workshop, con el alcance real del piloto ya definido.

Riesgo si no arrancamos esta semana: el cliente espera avances tangibles después del workshop. Si llegamos sin código nuevo de SAP, perdemos el momentum de la reunión del 13/05.